Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Deep Learning

Introduction to PyTorch Part 3

Lukas Galke & Md Shamim Ahmed – Slides from Lucas Dyssel

Spring 2025

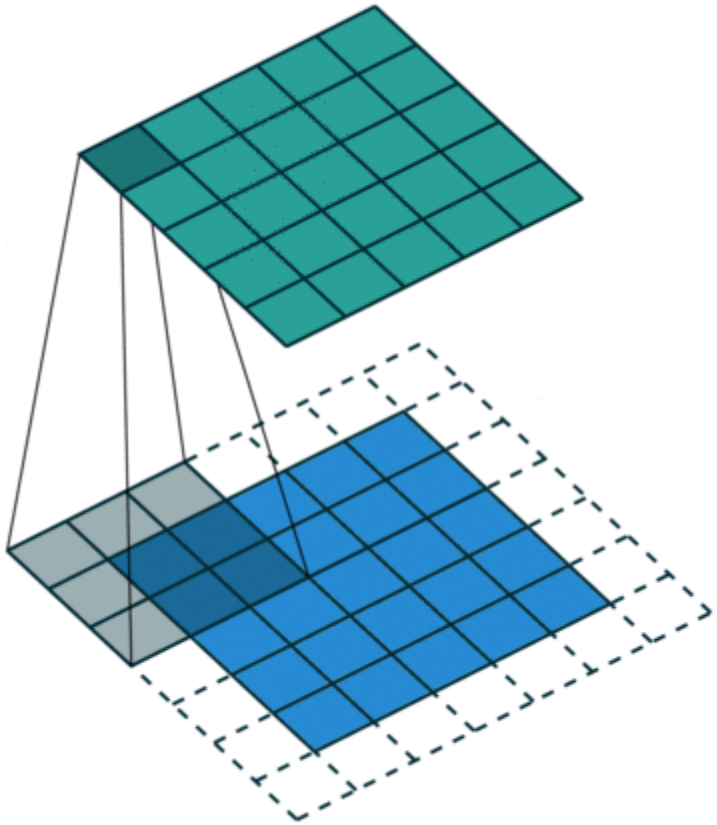
SDU

Keywords for this session

- Implementation of Convolution
- Implementation of Early stopping
- Practical considerations

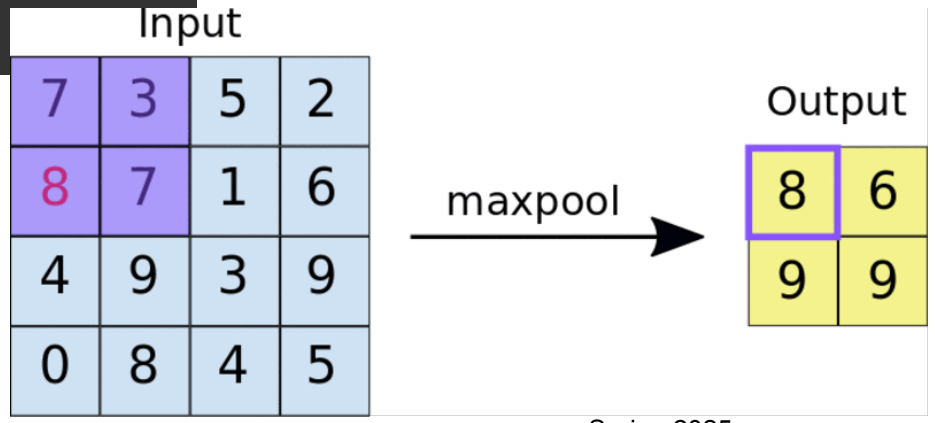
yTorch

```
self.conv1 = nn.Conv2d(
    in_channels = 64,       # N° of channels in the input image.
    out_channels = 32,      # N° of channels produced by the convolution.
    kernel_size = 3,        # Size of the convolving kernel.
    stride = 1,             # Modifies the amount of movement over the image.
    padding = 0,            # Padding added to all four sides of the input.
    dilation = 1,           # Spacing between kernel elements.
    groups = 1,             # N° of blocked connections from input to output.
    bias = True,            # If True, adds a learnable bias to the output.
    padding_mode = 'zeros’ # 'zeros', 'reflect', 'replicate' or 'circular'.
)
```



Defining MaxPooling Layer in PyTorch

```
self.mp = nn.MaxPool2d(
    kernel_size = 3,          # Size of the convolving kernel.

    stride = None,            # Modifies the amount of movement over the image.

    padding = 0,              # Padding added to all four sides of the input.

    dilation = 1,             # Spacing between kernel elements.

    return_indices = False,   # Returns the max indices along with the outputs.

    ceil_mode = False         # MaxPools over all positions by adding padding.

)
```



Defining CNN in PyTorch

```
class CNN(nn.Module):

    def __init__(self):
        super(CNN, self).__init__()                                          # [batch_size, channel, height, width]
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=64, kernel_size=3)   # Output size:([64, 64, 26, 26])
        self.maxpool1 = nn.MaxPool2d(kernel_size=2)                          # Output size:([64, 64, 13, 13])
        self.conv2 = nn.Conv2d(in_channels=64, out_channels=32, kernel_size=3)  # Output size:([64, 32, 11, 11])
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)                          # Output size:([64, 32, 5, 5])
        self.conv3 = nn.Conv2d(in_channels=32, out_channels=10, kernel_size=3)  # Output size:([64, 10, 3, 3])

        self.fc1 = nn.Linear(10 * 3 * 3, 128)      # n_input_channels == n_channels_conv * height * width
        self.dropout = nn.Dropout(0.2)
        self.fc2 = nn.Linear(128, 10)

        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.maxpool1(x)
        x = self.relu(self.conv2(x))
        x = self.maxpool2(x)
        x = self.relu(self.conv3(x))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

Data Augmentation

- **RandomResizedCrop**: Randomly crop and resize the image to a specified size.
- **RandomHorizontalFlip**: Randomly flip the image horizontally with a given probability.
- **RandomVerticalFlip**: Randomly flip the image vertically with a given probability.
- **RandomRotation**: Randomly rotate the image by a given angle range.
- **ColorJitter**: Randomly adjust brightness, contrast, saturation, and hue of the image.
- **RandomAffine**: Applies a random affine transformation to the image, including rotation, translation, scaling, and shearing.
- **RandomPerspective**: Applies a random perspective transformation to the image.
- **RandomGrayscale**: Randomly convert the image to grayscale with a given probability.
- **RandomErasing**: Randomly erase a rectangular region from the image, used for regularization and improving robustness.
- **RandomChoice**: Randomly apply one of the given transformations.

Data Augmentation

```
import torchvision.transforms as transforms
from torchvision.datasets import MNIST

transform = transforms.Compose([
    transforms.RandomAffine(degrees=10, translate=(0.1, 0.1), scale=(0.9, 1.1)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)),
])

train_dataset = MNIST(root='./data', train=True, download=True, transform=transform)
```

- **transforms.Compose([transform1, transform2, ..., transformN])**
  - Is used to create a transformation pipeline for data augmentation or preprocessing
- Using a transformation pipeline on a torchvision.datasets.MNIST dataset, we essentially

Data Augmentation

```
class CustomDataset(Dataset):
    def __init__(self, data, labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        image_path = self.data[idx]
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, label

custom_dataset = CustomDataset(file_paths, labels,
transform=transform)
```

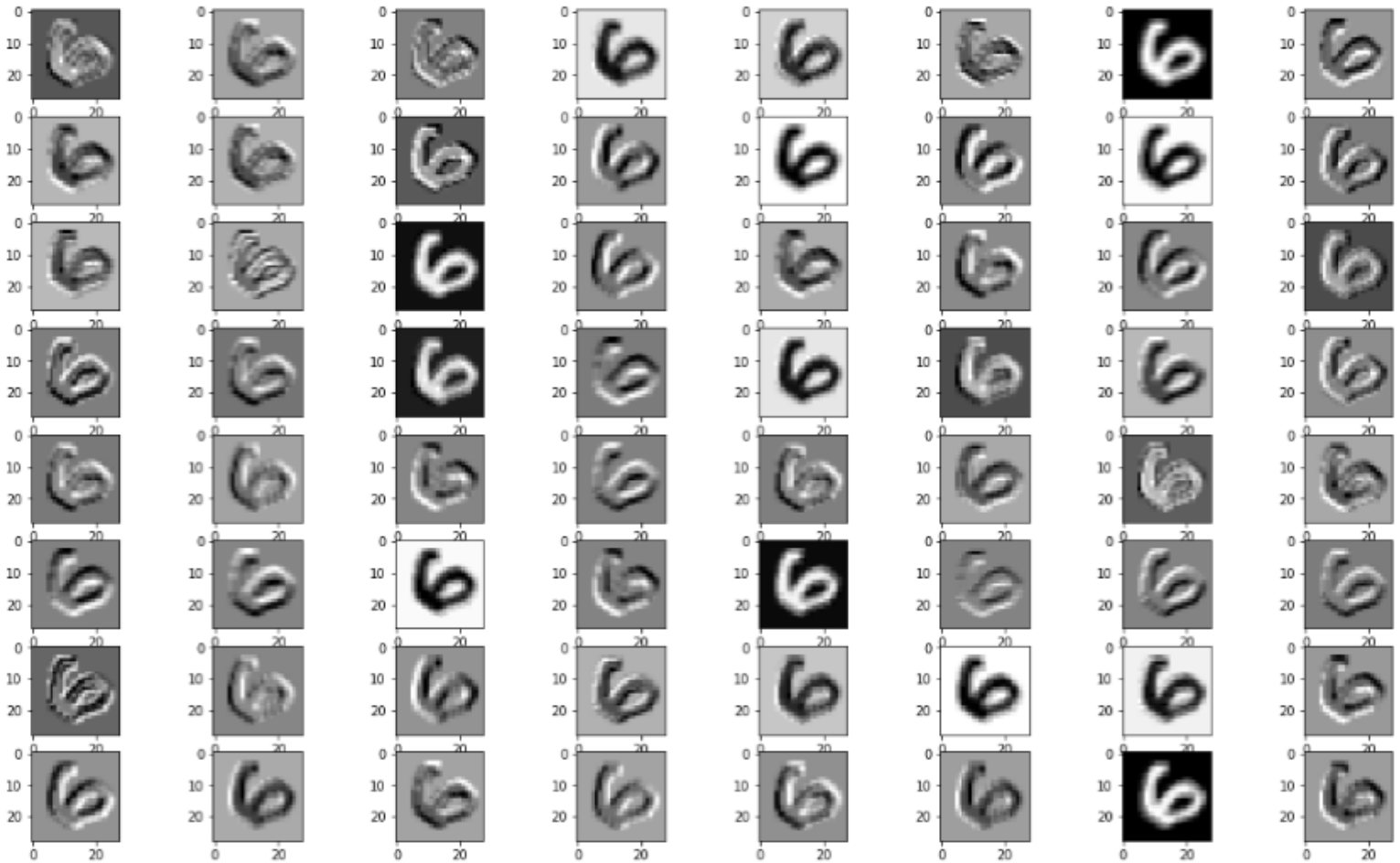
- **self.transform(image)** applies the specified transformations to the image
- When you then use the data loader on the custom dataset, it will apply the transformations to all images in the batch it loads
- Therefore, every time it load the same image, it will be different versions of it

Visualize Activations

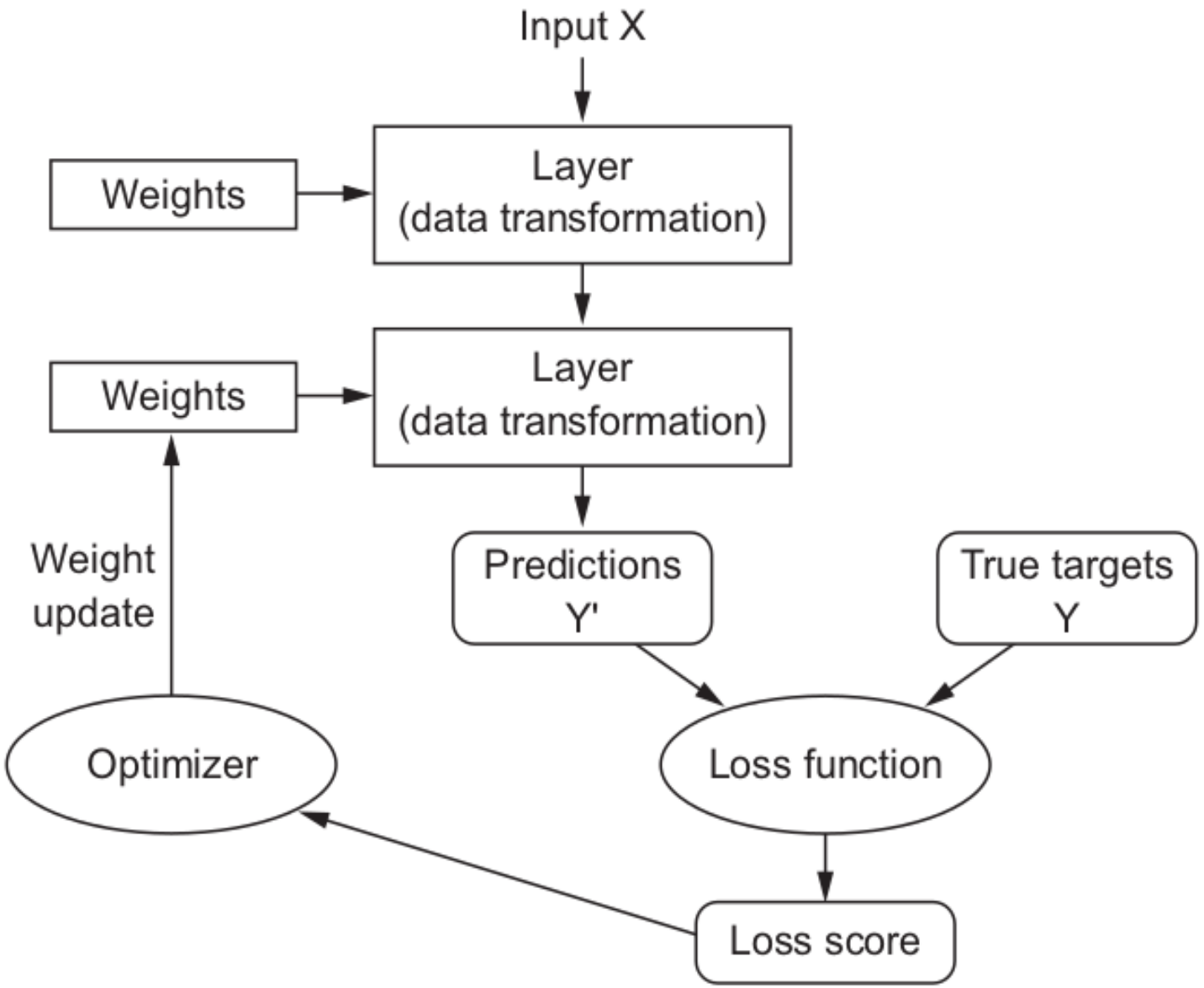
```
def save_first_feature_map(model, row_size, col_size):

    img_batch = next(iter(test_loader))[0]
    conv1_output = model.conv1(img_batch[0])
    layer_visualization = conv1_output.data

    for i, feature_map in
enumerate(layer_visualization):
        plt.subplot(row_size, col_size, i + 1)
        plt.imshow(feature_map.numpy(), cmap='gray')
        plt.axis('off')
    plt.savefig('outputs/feature_maps.png')
```

- **iter(test_loader)** converts it to an iterator which you can retrieve the first batch using **next().**
- Apply an enumerate on the layer's data to be able to index and plot the resulting activations.

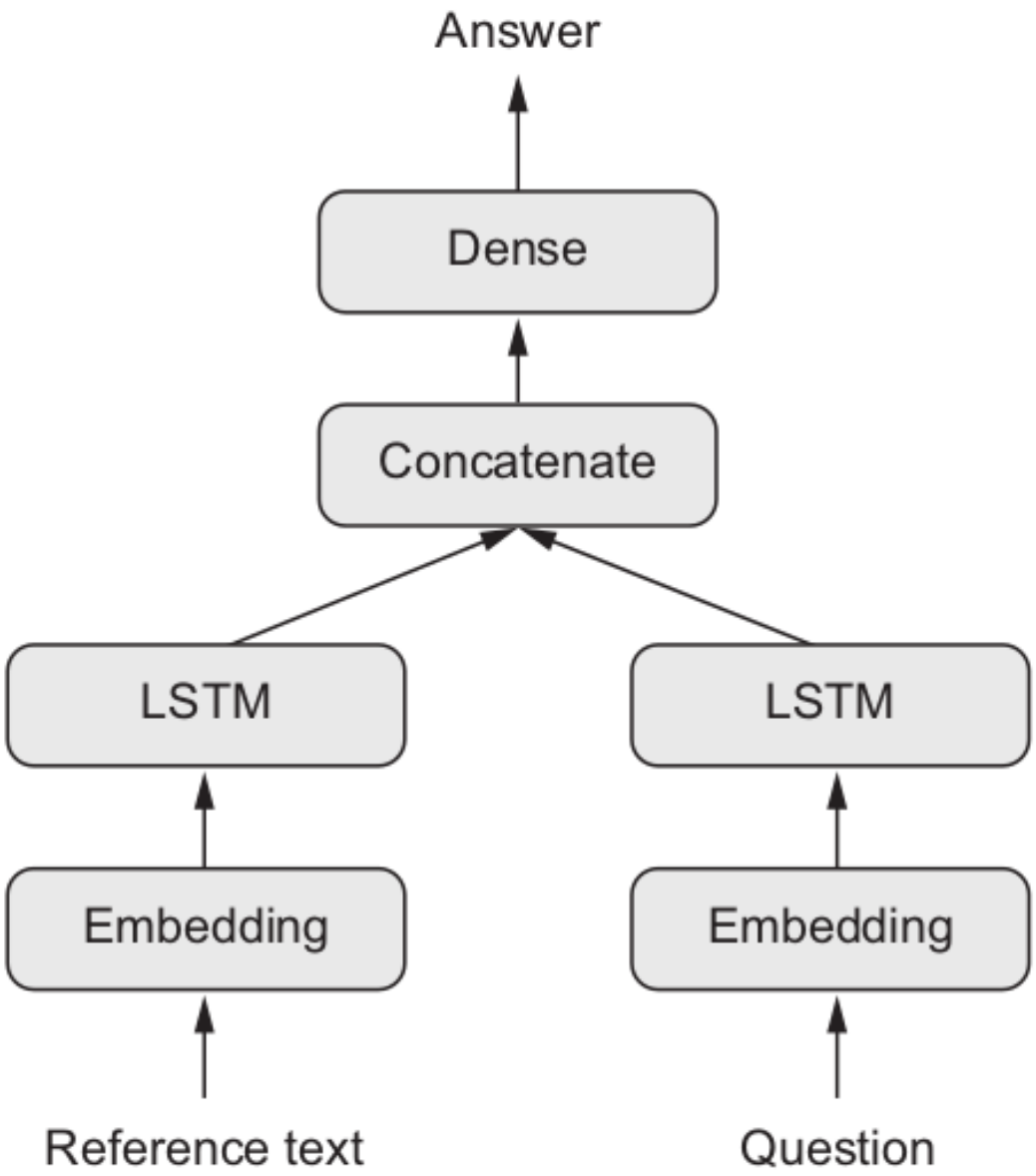
Visualize Activations



Overview of Training a Neural Network



Multi Input Models

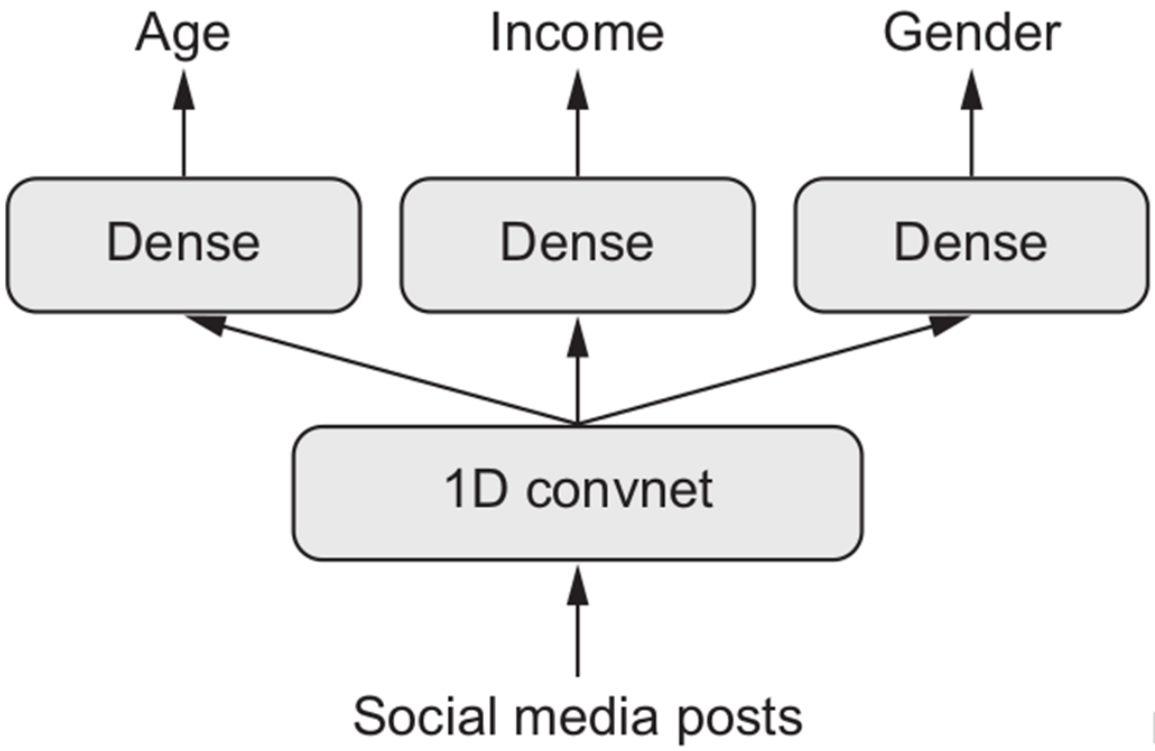


Multi Input Models

```
class NNetwork(nn.Module):
    def __init__(self):
        super(NNetwork, self).__init__()
        self.fc1 = nn.Linear(12, 16)
        self.fc2 = nn.Linear(16, 1)

    def forward(self, x1, x2):

        x = torch.cat((x1, x2), dim=1)

        x = torch.relu(self.fc1(x))
        x = torch.softmax(self.fc2(x))
        return x
```

- If your model takes two inputs, you simply concatenate the two input tensors along the second dimension
- Do however keep the dimensions in mind!
  - The input size of the first layer is no longer only dependent on 1 input, but 2
  - If the first input is of size 4 and the second input is of size 8, then the input size of the first fully connected layer should be $4 + 8 = 12$

Multi Output Models



Multi Output Models

```
class NNetwork(nn.Module):
    def __init__(self):
        super(NNetwork, self).__init__()
        self.fc1 = nn.Linear(10, 16)
        self.fc2 = nn.Linear(16, 1)
        self.fc3 = nn.Linear(16, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        output1 = torch.softmax(self.fc2(x))
        output2 = torch.sigmoid(self.fc3(x))
        return output1, output2
```

- If your model instead has two outputs, you simply add an extra “branch” with its own layers, activation function, etc.
- In this case **output1** uses **softmax** like before, but the new **output2** got its own fully connected layer and a sigmoid activation function

Early Stopping

```
best_loss = float('inf')
epochs_without_improvement = 0

###########################
###### training loop ######
###########################

# Early stopping
if loss < best_loss:
    best_loss = loss
    epochs_without_improvement = 0
else:
    epochs_without_improvement += 1

if epochs_without_improvement >= patience:
    print("Early stopping triggered.")
    exit()
```

- Before the training loop you want to initialize **best_loss** to very high value and **epochs_without_improvement** to 0
- After each epoch you want to check if your model improved by calculating the validation loss
  - This is the same as when we evaluate the model with respect to the test data, instead we just call “test” on the validation data each epoch rather than the end of the training
- If the loss is lower than the current **best_loss**, it means we should continue to train
- If it is not, we want to start counting
  - Once we reach some certain amount (in this case patience), we stop training and save the results
- This helps prevent overfitting and lowers wasted training time
- You can also use the package ignite to do early stopping

Using a Model for Predictions

```
import torchvision.models as models
from PIL import Image

model = models.resnet18(pretrained=True)
model.eval()

input_image = Image.open('path/to/your/
image.jpg')

with torch.no_grad():
    output = model(input_image)

predicted_idx = torch.argmax(output,
dim=1).item()
predicted_label = labels[predicted_idx]
```

- In this case we are loading the pretrained network ResNet18, you could also use your own trained network
- We then load an image using the PIL library
- We pass the image through ResNet18.
  - ResNet18 ends with a softmax, which means you get a probability distribution of what class the image belongs to
- We then use torch.argmax to get the index of the class which has the highest probability
- Lastly, we use this index to go through the list of classes the ResNet18 has trained on. This will allow us to find what class is in the image
  - Remember, a model can only predict on classes it has trained on. This is also the case for pretrained networks.